SCÉNÁŘ PŘEDNESU · KIV/PTE POKROČILÉ TESTOVÁNÍ · ZČU FAV

Coordinated omission

a pasti průměrných časů

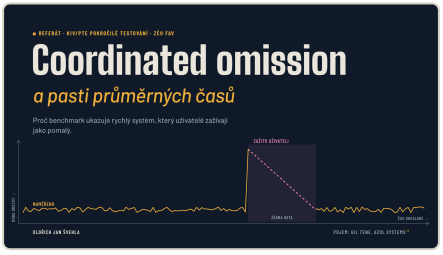
OLDŘICH JAN ŠVEHLA

15 SLIDŮ · PLÁN 9:45

ZVÝRAZNĚNÍ
Klíčová věta, kterou je dobré říct skoro doslova.

KLIK
▶ KLIK posune slide o krok (šipka → nebo mezerník).

ČAS
Kumulativní plán; v prezentaci ukazuje režim řečníka (klávesa P) i časovač.



01 0:00–0:55 · ~55 s

Úvod

Dobrý den. U zátěžového testu nás většinou nejvíc zajímá doba odezvy. A já vám chci ukázat, **že tohle číslo umí lhát, i když ho nikdo nefalšuje**. Důvody jsou dva. Prvnímu se říká **coordinated omission**, česky koordinované vynechání. Pojem zavedl Gil Tene z firmy Azul Systems. Ten druhý znáte všichni: **jsou to průměry**, tedy jak z tisíců měření uděláme jedno číslo. Graf na pozadí ukazuje celý problém ve zjednodušené podobě. Na vodorovné ose je čas odeslání požadavku, na svislé doba odezvy. Žlutá čára je to, co naměřil testovací nástroj: **jeden pomalý vzorek a pak během výpadku nic**. Růžově je to, co mezitím zažili uživatelé. Kdo přišel na začátku výpadku, čekal celou dobu, kdo přišel těsně před koncem, skoro nic.

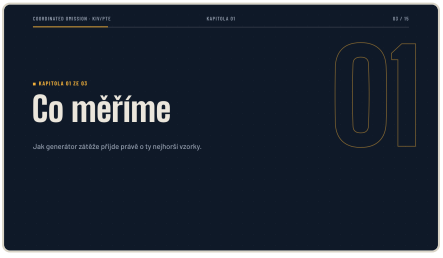


02 0:55–1:30 · ~35 s

Teze

Coordinated omission je **chyba v tom, co naměříte**. Měřicí nástroj se zasekne ve stejnou chvíli jako systém, a většina nejhorších okamžiků se tak do dat vůbec nedostane. To jsou ty prázdné kroužky na obrázku vlevo. Průměr je zase **chyba v tom, jak naměřené shrnete**. I kdyby byla data v pořádku, jedno číslo neřekne nic o nejpomalejších odpovědích, tedy o tom ocasu na obrázku vpravo. A právě ty si uživatelé pamatují.

▶ KLIK Zkráceně: **první chyba kazí data, druhá kazí závěry**.



03 1:30–1:35 · ~5 s

Předěl: Co měříme

Začnu tím měřením.



04 1:35–2:40 · ~65 s

Uzavřená smyčka

Většina nástrojů na zátěžové testy funguje **v takzvané uzavřené smyčce**. Třeba JMeter, open-source nástroj na zátěžové testy od Apache: v jeho běžné Thread Group hraje každé vlákno jednoho virtuálního uživatele. Vlákno pošle požadavek, počká na odpověď, zapíše si, jak dlouho to trvalo, a teprve potom pošle další. Horní řada ukazuje plán, třeba sto požadavků za sekundu. Dokud server odpovídá rychle, spodní řada ho přesně kopíruje.

▶ KLIK Teď se server na chvíli zasekne, třeba kvůli garbage collectoru nebo zámku v databázi. Požadavek, který zrovna běží, čeká, a vlákno čeká s ním. Když odpověď konečně přijde, zapíše se jeden pomalý vzorek.

▶ KLIK A tady je háček. Tyhle požadavky měly během zaseknutí odejít, jenže vlákno čekalo, a tak je nikdo neposlal. **A co nikdo neposlal, to nikdo nezměřil.** Z celého výpadku tak v datech zbude **jeden pomalý vzorek mezi stovkami rychlých**.



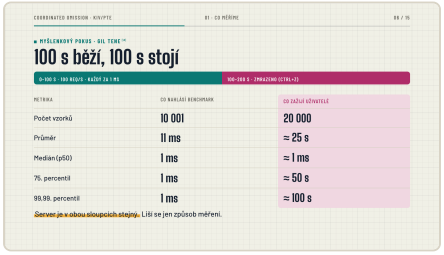
05 2:40–3:20 · ~40 s

Definice

Tene tomu říká coordinated omission, protože se měřený systém a měřicí nástroj nechtěně sladí: **když zpomalí systém, zpomalí i měření a přestane se ptát**.

▶ KLIK Představte si anketu spokojenosti, kde se ptáte jen lidí, kteří odcházejí od přepážky. Když se přepážka na hodinu zavře, nikdo neodchází, a tak si nikdo nestěžuje. Fronta venku se do výsledků vůbec nedostane.

▶ KLIK Test pak měří hlavně dobu obsluhy, service time: jak dlouho trvalo vyřídit požadavek, když na něj přišla řada. Jenže **uživatel zažívá dobu odezvy, response time, a to je čekání ve frontě plus obsluha**.



06 3:20–4:50 · ~90 s

Myšlenkový pokus: 100 s běží, 100 s stojí

Jak velký ten rozdíl může být? Tene má na to myšlenkový pokus. Systém sto sekund odpovídá za jednu milisekundu, sto požadavků za sekundu. Pak ho na sto sekund zmrazíme, v terminálu stačí Ctrl+Z. Nástroj v uzavřené smyčce nasbírá deset tisíc rychlých vzorků a jeden, který trvá sto sekund. Průměr mu vyjde necelých jedenáct milisekund. Připomenu, co je percentil: **99. percentil je hodnota, pod kterou leží 99 procent měření**, takže jen jeden požadavek ze sta je pomalejší. Náš pomalý vzorek je jeden z víc než deseti tisíc, a tak **i 99,99. percentil ukáže jednu milisekundu**. Vypadá to jako skvělý systém.

▶ KLIK Kdyby ale lidé chodili dál podle plánu, přišlo by během zmrazení dalších deset tisíc požadavků. Kdo přišel na začátku, čeká sto sekund, kdo na konci, skoro nic, **takže v průměru padesát sekund**. Polovina všech požadavků je rychlá a polovina čeká v průměru padesát sekund, proto **celkový průměr vychází asi na pětadvacet sekund**. 75. percentil padne doprostřed té pomalé poloviny, na padesát sekund, a 99,99. percentil skoro na sto. Medián leží přesně na hranici obou polovin, a tak zůstane kolem milisekundy.

▶ KLIK Všimněte si, že **server je v obou sloupcích stejný. Liší se jen způsob měření.**



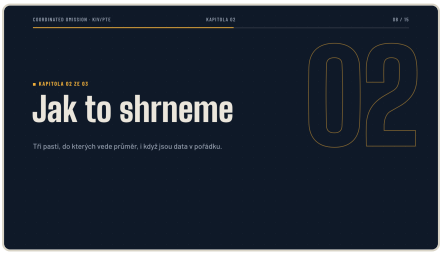
07 4:50–5:25 · ~35 s

wrk2: 200×

Není to jen teorie. Tene napsal wrk2, nástroj na zátěžové testy HTTP, který vypíše čísla s korekcí i bez ní. V příkladu z jeho dokumentace se server během třicetisekundového testu na 1,4 sekundy pozastaví. **Bez korekce vyjde 99. percentil šest milisekund.**

▶ KLIK **S korekcí 1 270 milisekund**, zhruba dvěstěkrát víc.

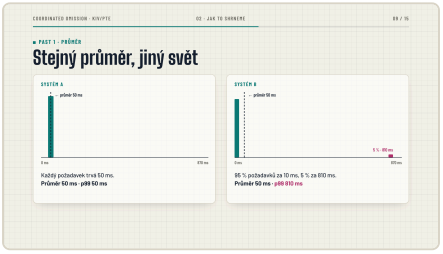
▶ KLIK Jediný rozdíl je v tom, odkud se počítá čas. Verze s korekcí ho **počítá od chvíle, kdy požadavek měl odejít**, ne od chvíle, kdy skutečně odešel.



08 5:25–5:35 · ~10 s

Předěl: Jak to shrneme

Řekněme, že už měříme správně. Data ale pořád musíme shrnout do čísla, které někomu ukážeme. A tady čekají tři pasti.

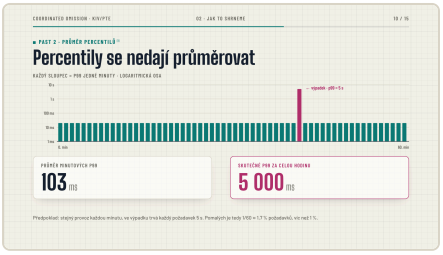


09 5:35–6:15 · ~40 s

Past 1: stejný průměr, jiný svět

První past je samotný průměr. Systém A odpoví na každý požadavek za padesát milisekund. Systém B odpoví v 95 procentech případů za deset milisekund a ve zbylých pěti procentech za 810. Když si to spočítáte, 0,95 krát 10 plus 0,05 krát 810 je přesně padesát. **Oba systémy mají stejný průměr.**

▶ KLIK Jenže **99. percentil systému B je 810 milisekund**, protože nejhorší procento leží v těch pomalých pěti procentech. To je šestnáctkrát víc než u systému A. **Proto se hlásí percentily a maximum, ne jedno číslo.**

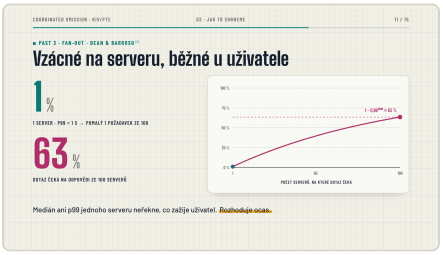


10 6:15–7:10 · ~55 s

Past 2: percentily se nedají průměrovat

Druhou past najdete skoro v každém dashboardu. Monitoring spočítá 99. percentil za každou minutu a pro hodinový pohled je zprůměruje. Tady je taková hodina: 59 minut má p99 dvacet milisekund a jedna minuta je výpadek, kdy každý požadavek trvá pět sekund. **Pozor, osa je logaritmická**, jinak by dvacet milisekund vedle pěti sekund nebylo vidět. Průměr minutových p99 vyjde 103 milisekund, tedy mírné zhoršení.

▶ KLIK Jenže při stejném provozu je ta minuta šedesátina všech požadavků, asi 1,7 procenta. To je víc než jedno procento, a proto **skutečný 99. percentil za hodinu je pět sekund**. Tene to říká přímo: **percentily se průměrovat nedají**. Když sčítáte přes čas nebo přes servery, **slučujte celé histogramy** a percentil spočítejte až z nich.



11 7:10–8:05 · ~55 s

Past 3: fan-out (The Tail at Scale)

Třetí past popsali Jeff Dean a Luiz Barroso z Googlu v článku The Tail at Scale: díváme se na jeden server, ale uživatel čeká na mnoho serverů najednou. Vezměte server, jehož 99. percentil je jedna sekunda, takže jeden požadavek ze sta trvá sekundu nebo víc.

▶ KLIK Jeden dotaz uživatele se ale ve velkém systému rozpadne na desítky až stovky paralelních volání a hotovo je, až když dorazí to nejpomalejší. Když dotaz čeká na sto takových serverů, šance, že budou všechny rychlé, je 0,99 na stou, asi 37 procent. Pravděpodobnost, že aspoň jeden bude pomalý, je tedy **jedna minus 0,99 na stou, tedy 63 procent**.

▶ KLIK Co je na jednom serveru vzácnost, je pro uživatele běžný zážitek. **Rozhoduje ocas.**



12 8:05–8:10 · ~5 s

Předěl: Jak měřit správně

Tak co s tím?



13 8:10–9:05 · ~55 s

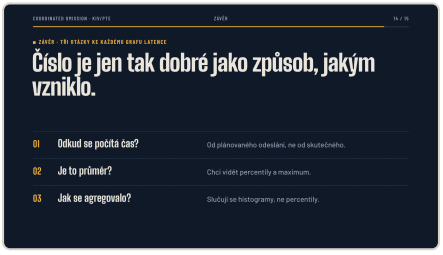
Řešení: dvě opravy měření, dvě opravy shrnutí

Opravy jsou dvě na měření a dvě na shrnutí. První: **posílat zátěž podle plánu, ne podle odpovědí**. Tomu se říká otevřený model. V JMeteru ho od verze 5.5 umí **Open Model Thread Group**, která neplánuje počet vláken, ale příchody požadavků. Podobně to umí nástroj k6 nebo wrk2.

▶ KLIK Druhá: **počítat čas od chvíle, kdy požadavek měl odejít**. Knihovna HdrHistogram, která ukládá latence do histogramu, umí chybějící vzorky i dopočítat.

▶ KLIK Třetí: místo jednoho čísla **hlásit rozdělení, tedy percentily a maximum**.

▶ KLIK A čtvrtá: při sčítání přes čas nebo servery **slučovat histogramy, ne percentily**. A jedna kontrola, kterou zvládnete hned: **sedí skutečná propustnost testu s plánovanou**? Propustnost je počet požadavků za sekundu. Pokud nástroj poslal méně požadavků, než měl, čekal.



14 9:05–9:35 · ~30 s

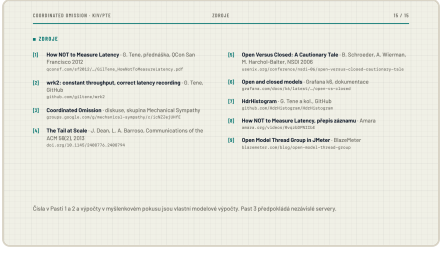
Závěr

Ve všech příkladech s měřením byl server pořád stejný. **Číslo je jen tak dobré jako způsob, jakým vzniklo.** Až příště uvidíte graf latence, zkuste si položit tři otázky.

▶ KLIK Odkud se počítá čas? Od plánovaného odeslání, nebo až od skutečného?

▶ KLIK Je to průměr? Pokud ano, chtějte percentily a maximum.

▶ KLIK A jak se data sčítala? Pokud někdo průměroval percentily, to číslo neplatí.



15 9:35–9:45 · ~10 s

Zdroje a poděkování

Zdroje najdete tady a celá prezentace je online i s odkazy. Děkuji za pozornost a rád odpovím na otázky.

PŘÍLOHA

Co si nastudovat

Tahle část se neříká nahlas. Je to podklad, abys rozuměl tomu, co stojí za slidy, a zvládl otázky po referátu.

1 · Pojmy v jedné větě

Doba odezvy response time	Čas od chvíle, kdy klient požadavek odeslal (správně: *měl* odeslat), do přijetí odpovědi. Zahrnuje **čekání ve frontě i obsluhu**.
Doba obsluhy service time	Čas, kdy server požadavek skutečně zpracovává. Tohle měří generátor zasažený coordinated omission.
Propustnost throughput	Počet dokončených požadavků za jednotku času (req/s).
Percentil pXX	Hodnota, pod kterou leží XX % měření. p50 = medián; p99 = jen 1 % požadavků je pomalejší.
Ocas rozdělení tail	Nejpomalejší odpovědi, typicky nad p99. Průměr a medián o něm nic neřeknou.
Uzavřený model closed	Pevný počet klientů (vláken); každý pošle další požadavek až po odpovědi, případně po pauze (think time). **Tempo příchodů závisí na serveru.**
Otevřený model open	Požadavky přicházejí podle plánu (rychlostí λ) **nezávisle na tom, jak rychle server odpovídá**. Odpovídá veřejnému webu nebo API.
JMeter · Thread Group	Apache JMeter je open-source nástroj na zátěžové testy. **Thread Group** je základní prvek testovacího plánu: určuje počet vláken (virtuálních uživatelů), náběh (ramp-up) a počet opakování nebo dobu běhu. Každé vlákno prochází plán postupně a **další požadavek pošle až po odpovědi na předchozí**, je to tedy uzavřený model.
wrk2 · k6	Nástroje na zátěžové testy. **wrk2** je Teneho úprava nástroje wrk pro HTTP: drží konstantní propustnost (`-R`) a měří od plánovaného času odeslání. **k6** (Grafana) píše testy v JavaScriptu a otevřený model nabízí přes executory `constant-arrival-rate` a `ramping-arrival-rate`.
Fan-out	Jeden dotaz uživatele se rozdělí na mnoho paralelních volání dalších serverů a odpověď je hotová, až když dorazí ta nejpomalejší. Proto se **vzácný ocas jednoho serveru stane běžným zážitkem uživatele**.
Coordinated omission	Systematická chyba měření: když systém zpomalí, generátor přestane posílat a měřit požadavky, takže **vynechá právě ty nejhorší vzorky**. Pojem zavedl Gil Tene (Azul Systems).
HdrHistogram	Knihovna, která zaznamenává latence do histogramu s velkým rozsahem hodnot a nastavitelnou přesností. Histogram jsou jen počty měření v jednotlivých intervalech, a proto **se histogramy dají sčítat**: sečtou se počty a percentil se spočítá až z výsledku. Percentily samotné takhle sčítat ani průměrovat nejde.

2 · Proč generátor „zpomalí s ním“

Pro uzavřený systém platí **zákon doby odezvy** (Response Time Law) z operační analýzy (Lazowska a kol., *Quantitative System Performance*, oddíl 3.3). V knize je zapsaný jako R = N/X – Z, tady upravený na propustnost:

$$X = N / (R + Z)$$

kde **X** je propustnost, **N** počet klientů (vláken), **R** doba odezvy a **Z** pauza mezi požadavky. Příklad: 200 vláken, Z = 1 s, R = 5 ms → X ≈ 199 req/s. Když se server zasekne a R vyroste, **X automaticky klesne**: generátor posílá méně požadavků přesně ve chvíli, kdy by měl systém zatěžovat nejvíc. To je mechanismus coordinated omission v jednom vzorci.

Zákon doby odezvy plyne z **Littleova zákona**, v knize N = X · R (často také L = λ · W): průměrný počet požadavků v systému = propustnost × průměrná doba v systému. Platí pro otevřený i uzavřený systém, pokud je v ustáleném stavu (kolik požadavků přijde, tolik jich odejde), a hodí se na rychlou kontrolu, jestli čísla z testu dávají smysl.

3 · Jak funguje korekce

- **wrk2** s parametrem `-R` drží konstantní propustnost. Každý požadavek má plánovaný čas odeslání a **latence se počítá od plánu**, ne od skutečného odeslání. Volbou `--u_latency` vypíše i nekorigovaná čísla pro srovnání.
- **HdrHistogram** `recordValueWithExpectedInterval(hodnota, interval)` : když je hodnota větší než očekávaný interval mezi požadavky, doplní navíc hodnoty *hodnota – interval*, *hodnota – 2·interval*, … dokud neklesnou pod interval. Příklad z myšlenkového pokusu: interval 10 ms (100 req/s), naměřeno 100 s → doplní se 9 999 hodnot od 99,99 s dolů po 10 ms. To jsou přesně požadavky, které generátor během výpadku nevyslal.
- Korekce je **odhad**, který předpokládá rovnoměrné příchody. Lepší je měřit rovnou v otevřeném modelu.

4 · Výpočty ze slidů

Myšlenkový pokus, benchmark	(10 000 × 1 ms + 1 × 100 000 ms) / 10 001 ≈ 11,0 ms. Tene v přednášce uvádí „10,9 ms“, přesně je to 10,999 ms.
Myšlenkový pokus, uživatelé	10 000 požadavků po 1 ms + 10 000 požadavků čekajících 100 s, 99,99 s, … 0,01 s (průměr ≈ 50 s) → (10 s + 10 000 × 50,005 s) / 20 000 ≈ 25 s. p75 = 15 000. hodnota ≈ 50 s, p99,99 ≈ 100 s.
wrk2	1 270 ms / 6,04 ms ≈ 210 → README uvádí „200x“.
Past 1	0,95 × 10 ms + 0,05 × 810 ms = 9,5 + 40,5 = 50 ms. p99 systému B leží v horních 5 %, tedy 810 ms.
Past 2	(59 × 20 ms + 5 000 ms) / 60 = 6 180 / 60 = 103 ms. Pomalá minuta = 1/60 ≈ 1,67 % požadavků > 1 % → skutečné p99 = 5 s.

Past 3	P(aspoň jeden pomalý z n) = 1 − 0,99^{n}. n = 10 → 9,6 %, n = 100 → 63,4 %. Předpoklad: servery jsou pomalé nezávisle na sobě.

5 · Možné otázky po referátu

Q · Je uzavřený model vždycky špatně?

> Ne. Pokud má reálný systém pevný počet klientů, kteří na odpověď opravdu čekají (dávkové zpracování, pevný počet terminálů), uzavřený model ho popisuje věrně. Problém je, když reálné příchody na rychlosti serveru nezávisí, jako u webu nebo veřejného API. Schroeder, Wierman a Harchol-Balter ukazují, že **volba modelu výrazně mění výsledky**, takže ji musí určovat realita, ne pohodlí nástroje.

Q · Nestačí v JMeteru přidat Constant Throughput Timer?

> Ne. Timer jen brzdí vlákna na cílovou propustnost. Když server stojí, vlákna dál čekají na odpovědi a nové požadavky neodcházejí. Víc vláken problém zmírní, ale neodstraní. Řešením je **Open Model Thread Group** (JMeter 5.5+) a kontrola skutečné propustnosti proti plánu.

Q · Proč nehlásit prostě maximum?

> Maximum je důležité a coordinated omission ho obvykle neschová, protože ten jeden pomalý vzorek se zapíše. Neřekne ale, *jak často* se to děje: jeden výpadek za den, nebo polovina požadavků? Proto **percentily a maximum dohromady**.

Q · Jak dostanu p99 za hodinu, když mám uložené jen minutové p99?

> Přesně nijak, z percentilů se zpět rozdělení nesložíš. Je potřeba ukládat **histogramy** (HdrHistogram, případně přibližné struktury jako t-digest), sloučit je a percentil spočítat až z výsledku.

Q · Proč se v myšlenkovém pokusu nezměnil medián?

> Polovina všech požadavků (prvních 100 s) je rychlá, takže medián leží přesně na hranici a zůstane na 1 ms. Medián o horší polovině neříká vůbec nic.

Q · Jak poznám coordinated omission ve výsledcích?

> Nejrychleji: **skutečná propustnost je nižší než plánovaná** (v JMeteru Throughput v Summary Reportu). Dále „díry" v časové ose vzorků v době, kdy systém zpomalil.

Q · Dá se s ocasem u fan-outu něco dělat?

> Dean a Barroso navrhují mimo jiné **hedged requests**: když odpověď nepřijde do očekávaného p95, pošle se stejný požadavek ještě jiné replice a použije se ta rychlejší odpověď. Zátěž to zvýší jen asi o 5 % a ocas výrazně zkrátí.

Q · Týká se to jen nástrojů na zátěžové testy?

> Ne. Vznikne všude, kde měřicí smyčka čeká na měřený systém: v mikrobenchmarcích, v klientovi, který si měří vlastní volání, nebo v monitoringu, který vidí jen dokončené požadavky.

6 · Doporučené studium

1. **Gil Tene: How NOT to Measure Latency** (přednáška, slidy z QCon SF 2012, přepis na Amara). Základ celého referátu: coordinated omission, percentily, HdrHistogram.
   Slidy: qconsf.com/sf2012/…/GilTene_HowNotToMeasureLatency.pdf
   Přepis: amara.org/videos/0vqzbDPNIIbE
2. **README nástroje wrk2**, část o coordinated omission a příklad s pozastavením serveru přes Ctrl+Z.
   github.com/giltene/wrk2
3. **J. Dean, L. A. Barroso: The Tail at Scale** (Communications of the ACM, 2013). Krátký článek o ocasu latence ve velkých systémech a technikách proti němu.
   doi.org/10.1145/2408776.2408794
4. **B. Schroeder, A. Wierman, M. Harchol-Balter: Open Versus Closed: A Cautionary Tale** (NSDI 2006). Kdy použít otevřený a kdy uzavřený model.
   usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale
5. **Dokumentace k6: Open and closed models**, stručné vysvětlení s příklady executorů.
   grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed
6. **BlazeMeter: Open Model Thread Group**, jak nastavit otevřený model v JMeteru.
   blazemeter.com/blog/open-model-thread-group
7. **README knihovny HdrHistogram**, záznam s očekávaným intervalem a slučování histogramů.
   github.com/HdrHistogram/HdrHistogram
8. **E. D. Lazowska, J. Zahorjan, G. S. Graham, K. C. Sevcik: Quantitative System Performance** (1984), kap. 3 Fundamental Laws, oddíl 3.3 (Littleův zákon a zákon doby odezvy). Autoři knihu zdarma zveřejnili online. Pro hlubší pochopení, proč se uzavřený systém sám přibrzdí.
   homes.cs.washington.edu/~lazowska/qsp

TAHÁK NA KARTIČKU

Měření: posílat podle plánu · čas od plánovaného odeslání · kontrolovat propustnost.

Shrnutí: percentily + maximum · nikdy neprůměrovat percentily · slučovat histogramy.

Tři otázky ke grafu: Odkud se počítá čas? Je to průměr? Jak se agregovalo?